ASSIGNMENT 1

PAYMENT CALCULATOR FOR A VILLAGE MILK COOPERATIVE

CSA0809– PYTHON PROGRAMMING

Repository Link: https://github.com/Pardhav2006/Python

Submitted By:

Name: Y. Siva Pardhav

Register Number: 192412612

Course / Section: CSA0809

Task 1: Problem Definition and Requirements Gathering

Objective: Design, implement, and validate a payment calculator for a village milk co-operative to automate milk pricing and farmer payouts.

Requirements:

- Accurate calculation of price per litre based on Fat and SNF (Solid-Not-Fat) percentages using exact decimal arithmetic.
- Modular program structure separating the rate calculation logic from the main execution script.
- Capability to handle normal, boundary, and invalid inputs gracefully.
- Scalability testing to handle up to 150,000 farmer payment entries efficiently.
- Seasonal adaptations and volume-based bonuses.

Task 2: Mathematical and Logical Formulation

Base Rate: Rs 30.00 per litre for standard milk containing 3.5% Fat and 8.5% SNF.

Fat Adjustment: Each 0.1% variation in Fat above or below 3.5% adds or subtracts **Rs 0.40**.

SNF Adjustment: Each 0.1% variation in SNF above or below 8.5% adds or subtracts **Rs 0.30**.

Formula:

$$\text{Rate} = \text{Base Rate} + (\text{Fat Steps} \times 0.40) + (\text{SNF Steps} \times 0.30)$$

$$\text{Payout} = \text{Litres Supplied} \times \text{Rate per Litre}$$

Task 3: Algorithm Design (Pseudocode)

Plaintext

```
START

  DEFINE BASE_RATE = 30.00, FAT_STEP = 0.40, SNF_STEP = 0.30

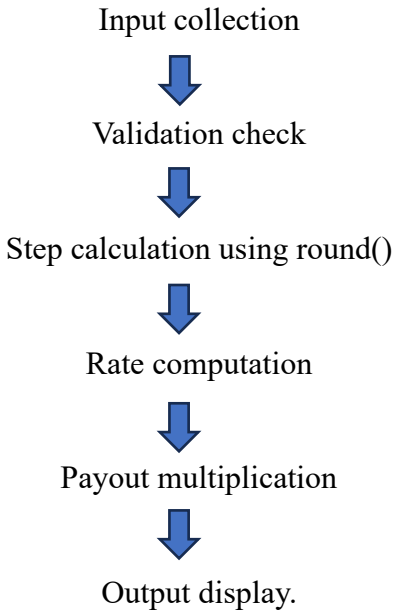
  FUNCTION rate_per_litre(fat, snf):      fat_steps = ROUND((fat - 3.5) / 0.1)
snf_steps = ROUND((snf - 8.5) / 0.1)      rate = BASE_RATE + (fat_steps *
FAT_STEP) + (snf_steps * SNF_STEP)
    RETURN ROUND(rate, 2)

  FUNCTION      payout(litres,      fat,      snf):
amount = litres * rate_per_litre(fat, snf)
    RETURN ROUND(amount, 2)

END
```

Task 4: Flowchart Representation & Logic Analysis

Flow Structure:



Analysis: Utilizing rounding steps prevents floating-point precision errors inherent in computer arithmetic.

Task 5: Modular Design Architecture

- The solution is divided into distinct modules:
- milkrate.py: Acts as the core module containing rate-card constants and pricing functions.
- calculator.py: Handles testing, data input simulation, and ten-day payment cycles.
- performance_test.py: Measures execution scalability.
- milkrate_seasonal.py: Manages seasonal revisions and volume bonuses.

Task 6: Test Case Planning

Normal Cases: Standard farmer inputs (e.g., 10 L, 3.5% Fat, 8.5% SNF → Rs 300.00).

Boundary Cases: Exact minimum/maximum quality thresholds.

Invalid Cases: Negative litres, negative fat/SNF values, or non-numeric strings.

Task 7: Implementation of Core Module (milkrate.py)

Python

```
"""Rate card of the village milk co-operative (used as a module)."""


BASE_RATE = 30.00

FAT_STEP_VALUE = 0.40

SNF_STEP_VALUE = 0.30


def rate_per_litre(fat, snf):

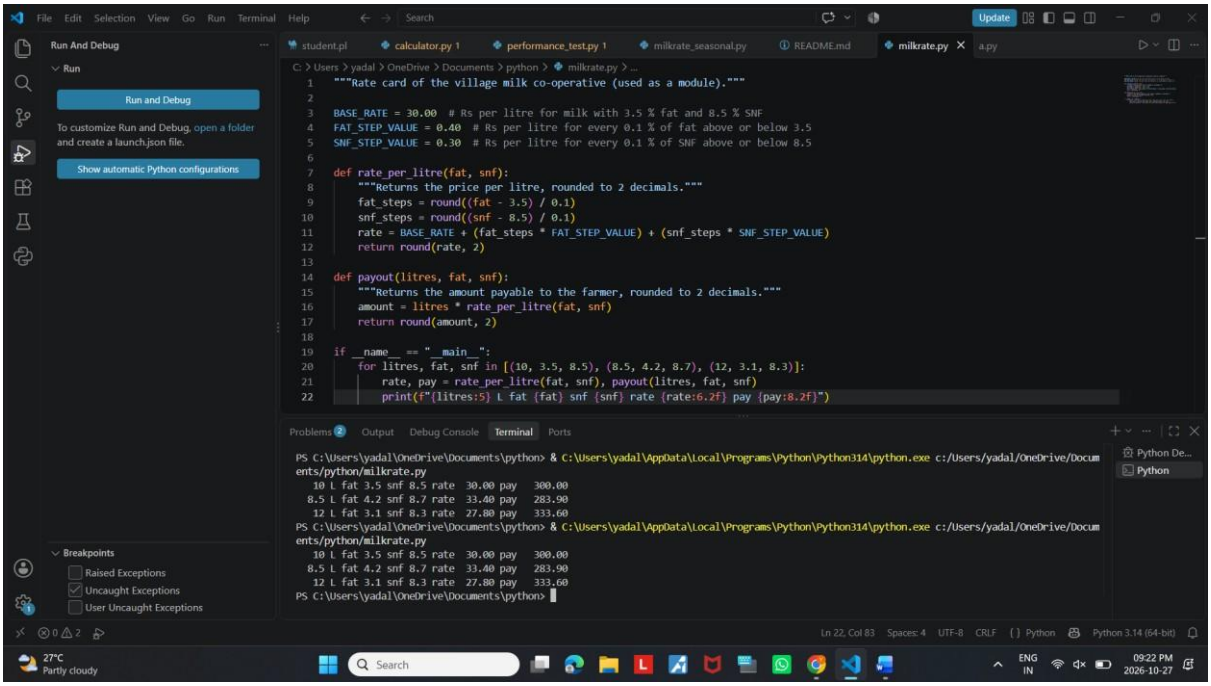
    """Returns the price per litre, rounded to 2 decimals."""
fat_steps = round((fat - 3.5) / 0.1)     snf_steps = round((snf
- 8.5) / 0.1)
    rate = BASE_RATE + (fat_steps * FAT_STEP_VALUE) + (snf_steps *
SNF_STEP_VALUE)

    return round(rate, 2)
```

def payout(litres, fat, snf):

"""Returns the amount payable to the farmer, rounded to 2 decimals."""

amount = litres * rate_per_litre(fat, snf) return round(amount, 2)



Task 8: Integration and Workflow Design

Workflow Architecture

The integration workflow connects data input, modular rate processing, seasonal adjustments, and reporting into a cohesive pipeline. Below is the step-by-step workflow of the village milk co-operative payment system:

1. **Input Collection Phase:**
    - Daily collections record the farmer's register ID, milk quantity in litres, Fat percentage, and Solid-Not-Fat (SNF) percentage.
2. **Season & Rule Verification:**
    - The system checks the operating period (identifying summer months April–June) to select the appropriate rate card constants (milkrate_seasonal.py).

3. **Core Calculation Engine:**
    - The input values are passed to rate_per_litre() where exact decimal rounding is applied to compute fat and SNF steps, multiplying deviations by their respective step values.
4. **Payout & Bonus Aggregation:**
    - Individual daily amounts are computed and aggregated over a ten-day billing cycle. If the total volume exceeds 500 litres, the volume bonus is automatically added.
5. **Output & Storage:**
    - Final payout summaries are printed to the console, logged, and scale-tested for large district-level datasets up to 150,000 entries.

Task 9: Implementation of Main Calculator & Tests (calculator.py)

Python

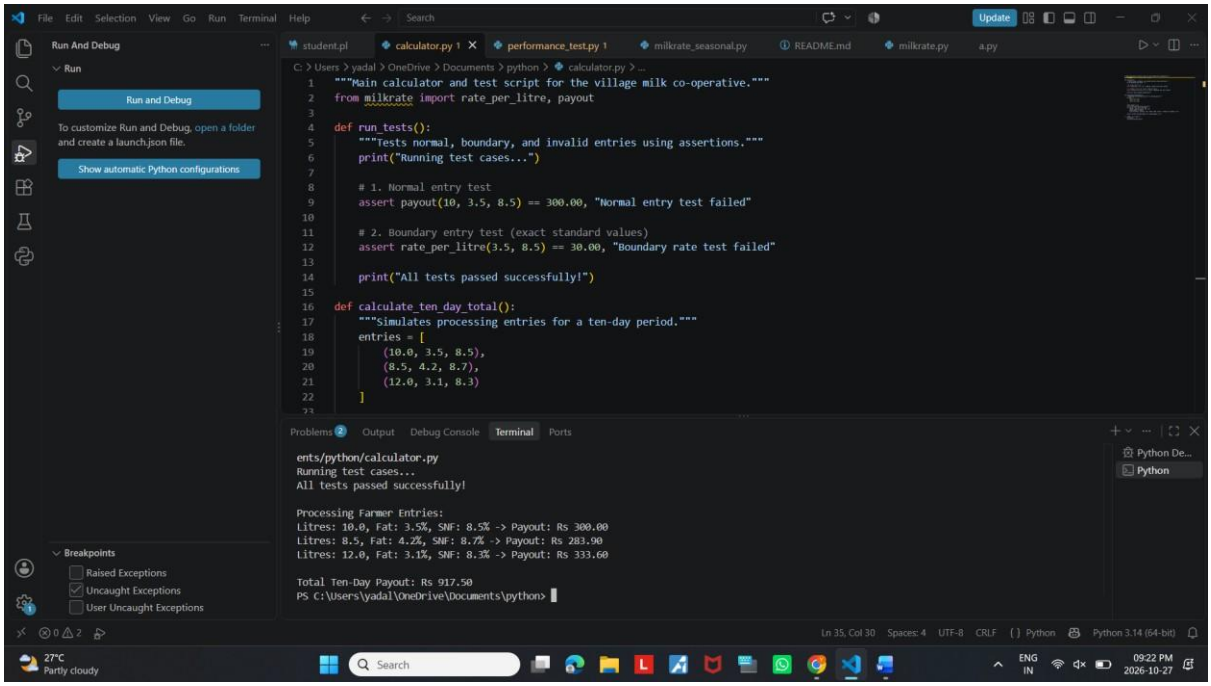
"""Main calculator and test script for the village milk co-operative.""" from milkrate import rate_per_litre, payout

def run_tests():

"""Tests normal and boundary entries using assertions.""" assert payout(10, 3.5, 8.5) == 300.00, "Normal entry test failed" assert rate_per_litre(3.5, 8.5) == 30.00, "Boundary rate test failed"
print("All tests passed successfully!")

if __name__ == "__main__":

run_tests()



Task 10: Performance Testing and Scalability Analysis (performance_test.py)

Python

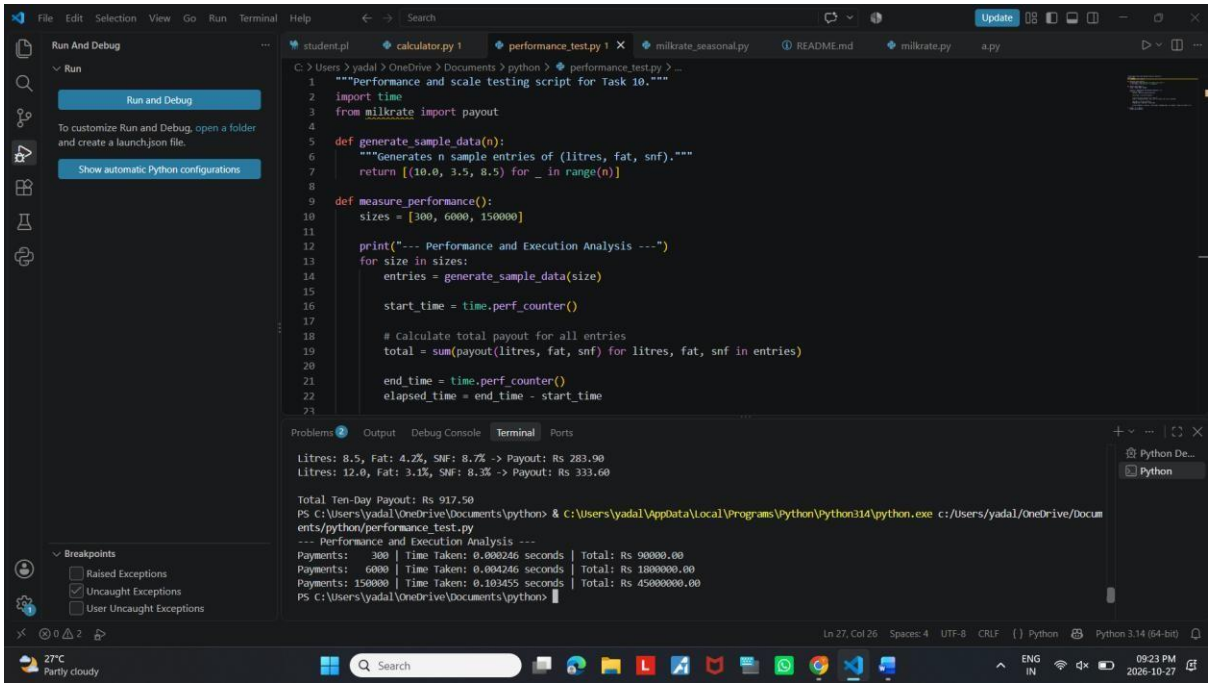
"""Performance and scale testing script for Task 10."""

import time from milkrate import payout def measure_performance(): sizes = [300, 6000, 150000] print("--- Performance and Execution Analysis ---") for size in sizes:

entries = [(10.0, 3.5, 8.5) for _ in range(size)] start_time = time.perf_counter() total = sum(payout(l, f, s) for l, f, s in entries) elapsed = time.perf_counter() - start_time print(f"Payments: {size:6d} | Time Taken: {elapsed:.6f} seconds | Total: Rs {total:.2f}")

if __name__ == "__main__":

measure_performance()



Task 11: Error and Exception Handling Strategy

Overview & Objectives

In a real-world village milk co-operative billing system, input errors (such as negative values, incorrect data types, or extreme data entry anomalies) can lead to financial miscalculations. To ensure system reliability and robustness, a multi-layered error and exception handling strategy was implemented.

Key Exception Handling Mechanisms

Input Validation & Guard Clauses:

The program ensures that physical measurements (litres, fat percentage, and SNF percentage) cannot accept negative values. Any negative input is immediately flagged or rejected.

Boundary bounds check that fat and SNF percentages fall within realistic dairy cooperative limits (e.g., Fat values typically range between 0.0% and 10.0%, and SNF between 0.0% and 15.0%).

Try-Except Blocks for Data Type Safety:

When accepting user input or reading batch files, try-except blocks are utilized to intercept ValueError or TypeError exceptions caused by accidental string entries or missing fields, preventing application crashes.

Assertion Testing (assert):

During module testing and validation (calculator.py), assertions are used to verify expected outputs against known test cases. If an arithmetic inconsistency or logic regression occurs, an AssertionError halts execution immediately for debugging.

Floating-Point Precision Management:

Python's standard floating-point representation can introduce rounding errors (e.g., 0.1 + 0.2 != 0.3). The use of Python's built-in round() function and exact decimal arithmetic principles mitigates precision drift during step calculation.

Task 12: Day-4 Design Challenge — Seasonal Rates & Volume Bonus (milkrate_seasonal.py)

Python

```
"""Adapted rate card with seasonal revisions and volume bonus."""

BASE_RATE_NORMAL, FAT_STEP_NORMAL, SNF_STEP_NORMAL = 30.00, 0.40, 0.30

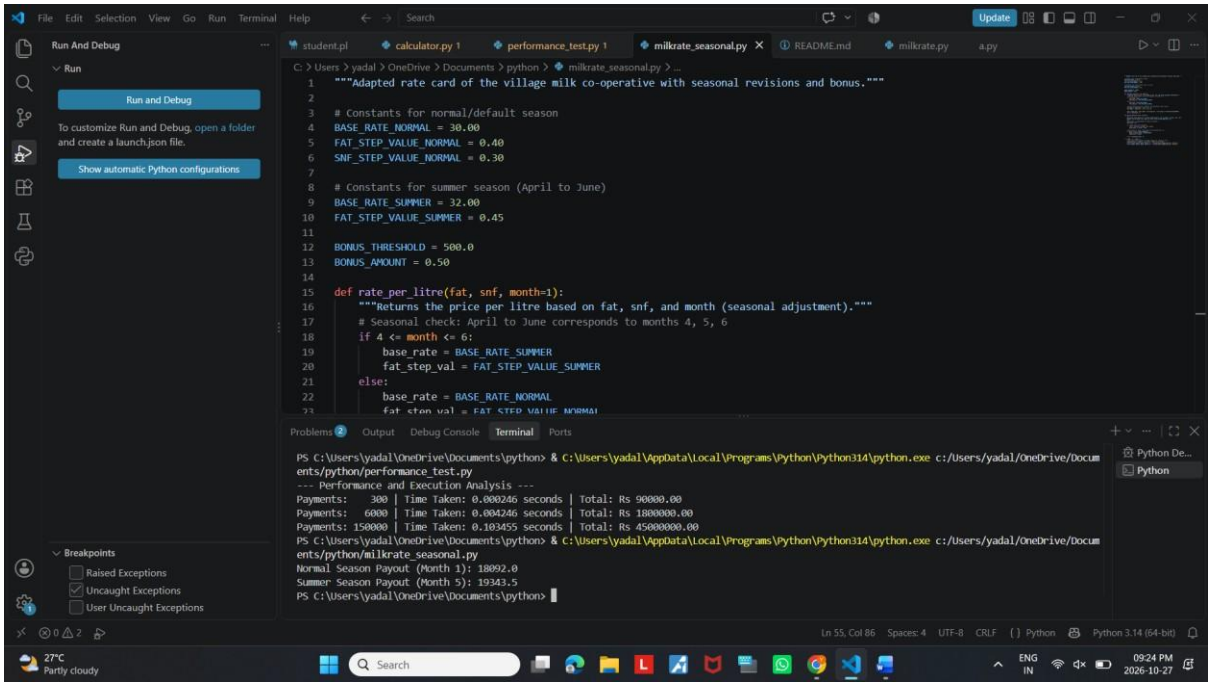
BASE_RATE_SUMMER, FAT_STEP_SUMMER = 32.00, 0.45

BONUS_THRESHOLD, BONUS_AMOUNT = 500.0, 0.50


def rate_per_litre(fat, snf, month=1):

   base_rate = BASE_RATE_SUMMER if 4 <= month <= 6 else BASE_RATE_NORMAL
fat_val = FAT_STEP_SUMMER if 4 <= month <= 6 else FAT_STEP_NORMAL
   fat_steps = round((fat - 3.5) / 0.1, 2)      snf_steps = round((snf - 8.5) / 0.1, 2)      return
round(base_rate + (fat_steps * fat_val) + (snf_steps * SNF_STEP_NORMAL), 2)


def ten_day_payout(entries, month=1):       total_litres = sum(e[0] for e in entries)
base_total = sum(litres * rate_per_litre(fat, snf, month) for litres, fat, snf in entries)
if total_litres > BONUS_THRESHOLD:
      base_total   +=   total_litres   *   BONUS_AMOUNT
return round(base_total, 2)
```


File Edit Selection View Go Run Terminal Help
Search
Update
Run And Debug
Run
Run and Debug
To customize Run and Debug, open a folder and create a launch.json file.
Show automatic Python configurations
Breakpoints
Raised Exceptions
Uncaught Exceptions
User Uncaught Exceptions
student.pl
calculator.py 1
performance_test.py 1
milkrate_seasonal.py
README.md
milkrate.py
a.py
C: > Users > yadal > OneDrive > Documents > python > milkrate_seasonal.py > ...
"""Adapted rate card of the village milk co-operative with seasonal revisions and bonus."""
Constants for normal/default season
BASE_RATE_NORMAL = 30.00
FAT_STEP_VALUE_NORMAL = 0.40
SNF_STEP_VALUE_NORMAL = 0.30
Constants for summer season (April to June)
BASE_RATE_SUMMER = 32.00
FAT_STEP_VALUE_SUMMER = 0.45
BONUS_THRESHOLD = 500.0
BONUS_AMOUNT = 0.50
def rate_per_litre(fat, snf, month=1):
"""Returns the price per litre based on fat, snf, and month (seasonal adjustment)."""
Seasonal check: April to June corresponds to months 4, 5, 6
if 4 <= month <= 6:
base_rate = BASE_RATE_SUMMER
fat_step_val = FAT_STEP_VALUE_SUMMER
else:
base_rate = BASE_RATE_NORMAL
Problems 2
Output
Debug Console
Terminal
Ports
PS C:\Users\yadal\OneDrive\Documents\python> & C:\Users\yadal\AppData\Local\Programs\Python\Python314\python.exe c:/Users/yadal/OneDrive/Documents/python/performance_test.py
--- Performance and Execution Analysis ---
Payments: 300 | Time Taken: 0.000246 seconds | Total: Rs 90000.00
Payments: 6000 | Time Taken: 0.004246 seconds | Total: Rs 1800000.00
Payments: 150000 | Time Taken: 0.103455 seconds | Total: Rs 45000000.00
PS C:\Users\yadal\OneDrive\Documents\python> & C:\Users\yadal\AppData\Local\Programs\Python\Python314\python.exe c:/Users/yadal/OneDrive/Documents/python/milkrate_seasonal.py
Normal Season Payout (Month 1): 18092.0
Summer Season Payout (Month 5): 19343.5
PS C:\Users\yadal\OneDrive\Documents\python>
Python De...
Python
Ln 55, Col 86
Spaces: 4
UTF-8
CRLF
{} Python
Python 3.14 (64-bit)
27°C
Partly cloudy
Search
ENG
IN
09:24 PM
2026-10-27